

A Simulation-Based Reproduction of Comparative Sensor Fusion Algorithms on a Miniature IMU

*Replicating Badia et al. (ISITIA 2023) with a Synthetic MPU-6050 Pipeline
and an Analytical ATmega328P Timing Model*

Abdullah Altepe · Mehmet Bahçeci · Yusuf Oruç · Murat Demir
Department of Electrical and Electronics Engineering
Marmara University · 2026

Outline

What we will cover, in seven phases

I

The Problem

Orientation, IMUs, and why each sensor fails alone.

II

The Reference Paper & Our Approach

What Badia et al. did; why we went simulation-only.

III

Algorithm 1: Trigonometric

concept · math · code · result · evaluation

IV

Algorithm 2: Kalman Filter

concept · math · code · result · evaluation

V

Algorithm 3: Madgwick Filter

concept · math · code · result · evaluation

VI

Algorithm 4: Mahony Filter

concept · math · code · result · evaluation

VII

Side-by-Side & Reproduction Verdict

Five claims tested; one surprising divergence.

What is “Orientation”?

Three angles describe how an object is rotated in 3-D space

When you tilt your phone, three things can change:

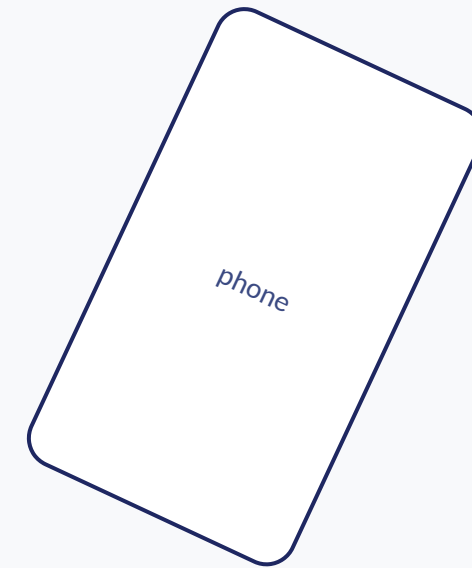
Roll rotation around the long axis (tilting sideways)

Pitch rotation around the short axis (tilting forward/back)

Yaw rotation around the vertical axis (turning left/right)

We estimate **roll and pitch only** — the MPU-6050 has no magnetometer, so yaw drifts uncorrected. Same scope as the reference paper.

A simple example



Held flat: roll = 0°, pitch = 0°

Tilted right: roll = +25°

Tipped forward: pitch = +25°

Where Do We Need This?

Orientation estimation appears almost everywhere in modern engineering



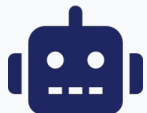
Drones

A quadcopter has to know it is level before it can hover, turn, or land safely.



VR Headsets

If the headset cannot track head rotation in real time, the virtual world feels jittery and motion sickness follows.



Self-Balancing Robots

A two-wheel robot stays upright only as long as it knows exactly which direction it is leaning.

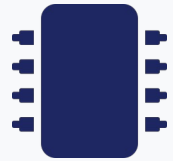


Wearables

Smart-watches use orientation to count steps, classify activities, and detect falls.

The Target Hardware

An MPU-6050 sensor on an Arduino Nano — the cheapest realistic platform



InvenSense MPU-6050

the IMU — the sensor that does the measuring

Approximate cost US \$3

Tri-axis accelerometer ± 2 g default

Tri-axis gyroscope $\pm 250^\circ/\text{s}$ default

Bus I²C, up to 400 kHz

Power 3.3 V

What it actually is: *a rice-grain-sized chip with two micro-mechanical sensors and an I²C interface — sold on a 2 cm breakout board.*



Arduino Nano (ATmega328P)

the brain — runs the algorithm in real time

Architecture 8-bit AVR

Clock 16 MHz

RAM 2 KB

Flash 32 KB

Approximate cost US \$5 (clone)

Compare with: *a modern phone has ~1 million times more compute. Algorithms must be tiny enough to run inside this budget at 100 Hz.*

The Two Sensors Inside the IMU

Each one alone is incomplete — the trick is in their characters

Accelerometer

“Which way is gravity pulling me right now?”

- ✓ Stable in the long run — no drift
- ✗ Very noisy in the short run
- ✗ Confused by any vibration or shake

Analogy: *like a friend who always knows which way is down, but who twitches a lot when answering.*

Gyroscope

“How fast am I spinning right now?”

- ✓ Smooth and responsive in the short run
- ✗ To get an angle we must integrate over time
- ✗ Tiny bias → error grows linearly forever (drift)

Analogy: *like a friend with a great sense of motion but a bad memory — every minute they drift further from the truth.*

Why Neither Sensor Alone Is Enough

Two failure modes, both real, both visible in our experiments

Accelerometer alone → noise

Each accelerometer sample is contaminated by Gaussian noise (~ 0.026 m/s² in our model). When we apply atan2 to recover the angle, that noise leaks straight through.

Effect: the angle estimate jitters by roughly $\pm 0.5^\circ$ every sample, even though the sensor is mechanically still.

Measured: $\sigma_{\text{Trig}} = 0.154^\circ$ ($\approx 4\times$ larger than Mahony)

Gyroscope alone → drift

A gyro sample is much cleaner than an accelerometer sample. But a real gyro has a constant bias ($\sim 0.5^\circ/\text{s}$ in our model). Integrating that bias accumulates without bound.

Effect: after 1 minute the angle is wrong by 30° ; after 10 minutes, by $\sim 300^\circ$. A still sensor looks like it has spun a full turn.

This is why we never use the gyro alone for absolute angle.

The Idea: Sensor Fusion

Combine the two sensors so each one's strength covers the other's weakness

The fusion idea, written like an equation:



What changes between fusion algorithms? Only the answer to one question: “how should I weight the accelerometer evidence against the gyroscope evidence at every step?” The four algorithms in the next slide give four different mathematical answers.

The Reference Paper

What Badia et al. did, and what they claimed

Sansone, Bartolini, Perin, Badia — “A Comparative Analysis of Sensor Fusion Algorithms for Miniature IMU Measurements,” *ISITIA 2023, IEEE*.

What they did

- Wired an MPU-6050 to an Arduino Nano.
- Implemented the four algorithms directly on the Arduino.
- Held the sensor at known $\pm 15^\circ$ tilts.
- Measured (a) the accuracy of each estimate vs. the known tilt and (b) the per-step compute time on the Arduino.

Their main findings

- Trigonometric $\rightarrow$ noisiest output.
- Mahony $\rightarrow$ fastest filter per step.
- Kalman $\approx$ Madgwick in compute time.
- Kalman $\rightarrow$ most accurate.
- *Headline: “accuracy $\rightarrow$ Kalman; speed $\rightarrow$ Mahony; Madgwick in between.”*

Our Challenge: No Hardware

We chose a fully simulated reproduction — here is why



The constraint

We had no MPU-6050 module and no Arduino Nano on hand at the time of writing.

Two options:

A. Order hardware, build a mechanical $\pm 15^\circ$ fixture, run the experiment (~2 weeks).

B. Build a synthetic data pipeline that mimics the MPU-6050 and an analytical timing model for the ATmega328P (~1 week).

We chose B.



We chose option B — a simulation reproduction

Two real benefits

- Ground-truth orientation is exact — no mechanical fixture error to subtract.
- The whole experiment is deterministic — anyone with Python can re-run it bit-for-bit.

One real cost

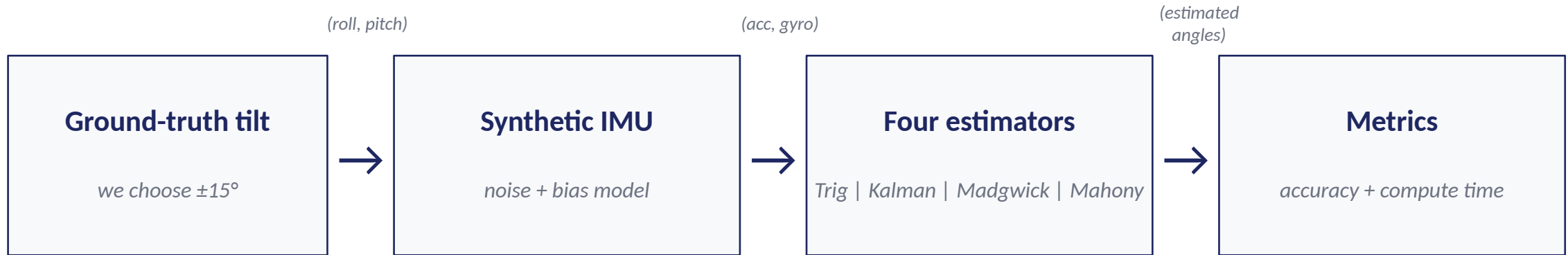
- Synthetic Gaussian noise does not capture every aspect of real silicon (temperature drift, vibration sensitivity, 1/f flicker).

If hardware is added later, the same code base absorbs real CSV logs without structural changes.

Our Simulation Pipeline

End-to-end: ground-truth tilt → noisy synthetic IMU → four algorithms → metrics

Read this left-to-right — each block feeds the next.



Experiment specification

Sample rate 100 Hz | **Segments** 4 ($\pm 15^\circ$ roll & pitch) | **Duration** 30 s each | **Total** 12,000 samples

Same noise model and bias realisation across all four segments — like a single physical sensor.

Synthetic IMU: Noise & Bias Model

Built to match the MPU-6050 datasheet at default ranges

Parameter	Value	Source
Accelerometer noise σ	0.026 m/s ²	datasheet @ 44 Hz BW
Gyroscope noise σ	0.05 °/s	datasheet
Accelerometer bias σ	0.05 m/s ²	typical, drawn once per session
Gyroscope bias σ	0.5 °/s	typical, drawn once per session
Sample rate	100 Hz	matches reference paper
Accel range	±2 g	default
Gyro range	±250 °/s	default

How the data is generated

1. Pick a known tilt (e.g. roll = +15°).
2. Project gravity into the body frame analytically — this gives the noise-free accelerometer reading.
3. Add Gaussian noise + a constant bias to both sensors.
4. The gyroscope is zero-mean (the sensor is static) plus its own bias and noise.
5. Repeat 100 times per second for 30 s → 3,000 samples.

Because we generate the data, the ground-truth tilt is known to infinite precision — perfect for accuracy comparison.

Experimental Protocol & Success Criterion

What we measure, and what counts as a successful reproduction

Protocol

1. Run all four algorithms on the same four $\pm 15^\circ$ segments.
2. Discard the first 5 s of each segment to let the filters converge.
3. On the remaining 25 s, compute $|\mu|$ (mean error vs. ground truth) and σ (standard deviation of the error).
4. Average across the four segments $\rightarrow$ one row per algorithm.
5. Compute per-step time analytically from operation counts $\times$ ATmega328P cycle costs.

Reproduction is successful if...

- C1.** σ_{Trig} is the largest in the table.
- C2.** Kalman and Madgwick agree within 20 % in compute time.
- C3.** Mahony is faster than both Kalman and Madgwick.
- C4.** Kalman is among the lowest- σ algorithms.
- C5.** *Trig is the most expensive method overall (implicit).*

Trigonometric Method — Concept

The unfused baseline. Just the accelerometer + atan2.

Algorithm 1 of 4 • Trigonometric

What it does

When the IMU is static, the accelerometer reads gravity projected onto its three axes. From that projection we recover roll and pitch directly using trigonometry — no fusion, no filtering, no state.

Why we include it: every fancy filter has to beat this baseline. If it does not, the extra complexity is not earning its place.

Pros and cons

- ✓ Trivial to implement (3 lines of code).
- ✓ Zero drift — always anchored to gravity.
- ✗ Passes every bit of accelerometer noise into the angle.
- ✗ Useless under vibration or motion.
- ✗ No yaw — gravity tells you nothing about heading.

Trigonometric Method — Math

Two `atan2` calls, that is the whole algorithm

Algorithm 1 of 4 • Trigonometric

$$\varphi = \text{atan2}(a_y, a_z)$$

$$\theta = \text{atan2}(-a_x, \sqrt{a_y^2 + a_z^2})$$

Reading the formulas:

- a_x , a_y , a_z are the three accelerometer channels.
- φ (roll) is the angle the gravity vector makes with the body's z-axis in the y-z plane.
- θ (pitch) tilts the gravity vector out of that plane along x.
- `atan2` is a four-quadrant arctangent — it returns the correct sign automatically.

No state, no memory, no parameters to tune. Each estimate is computed from scratch on every sample.

Trigonometric Method — Code

From algorithms.py — the entire estimator in five lines

Algorithm 1 of 4 • Trigonometric

```
class TrigEstimator:
    def update(self, acc):
        ax, ay, az = acc
        roll = atan2(ay, az)
        pitch = atan2(-ax, sqrt(ay*ay + az*az))
        return roll, pitch
```

Notes

- No `__init__` — there is no state to keep.
- Same code, line-for-line, on the Arduino in C++.
- Total: 6 lines.

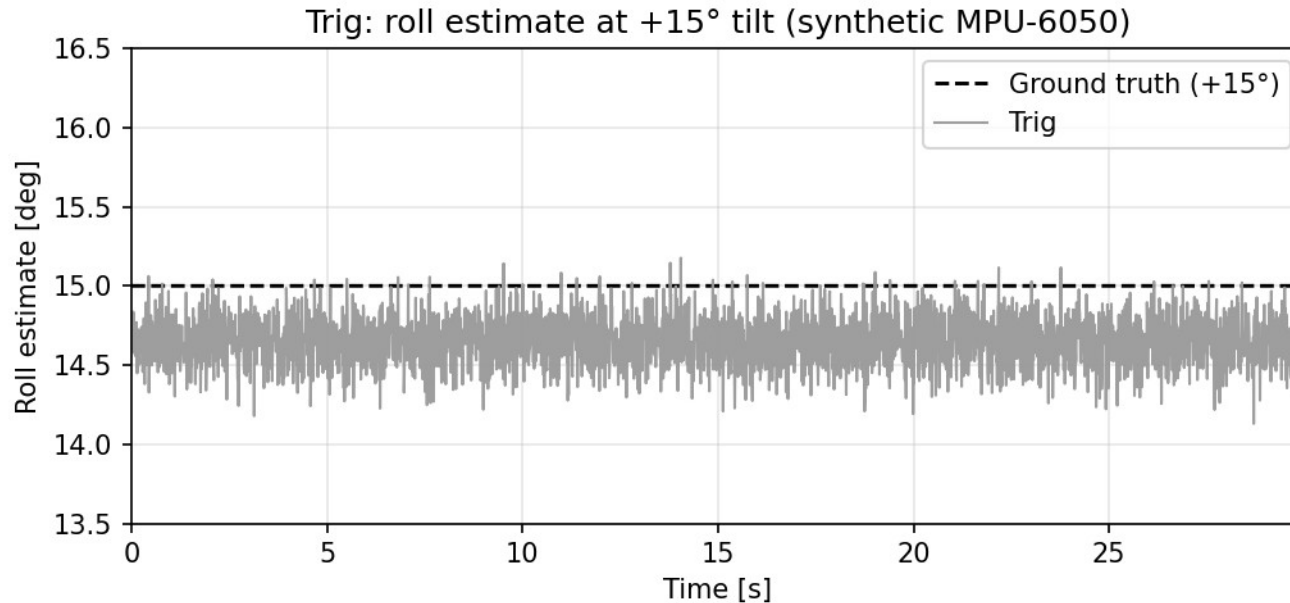
Per-step instruction count (used by the timing model)

$2 \times \text{atan2} + 1 \times \text{sqrt} + 3 \times \text{fmul} + 1 \times \text{fadd}$
 $\approx 9,090 \text{ cycles} \approx 568 \mu\text{s} / \text{step}$ on the ATmega328P @16 MHz.

Trigonometric Method — Result & Evaluation

Roll estimate at +15° tilt, plus accuracy numbers averaged over the four segments

Algorithm 1 of 4 • Trigonometric



Accuracy

$|\mu|$ roll = 0.302° σ roll = 0.154°
 $|\mu|$ pitch = 0.086° σ pitch = 0.152°

Compute time (estimated)

568 μ s / step
max sustainable rate $\approx 1,760$ Hz

Take-away. Trig confirms its role as the noisy baseline. The mean offset ($\sim 0.3^\circ$) is just the bias floor — every algorithm sees it because every algorithm uses the same biased accelerometer. The σ of 0.154° is the largest in the comparison and roughly 4× larger than Mahony's.

Kalman Filter — Concept

Predict with the gyroscope, correct with the accelerometer — weighted by uncertainty

Algorithm 2 of 4 • Kalman Filter

PREDICT

Use the gyroscope to push the angle estimate forward in time. Uncertainty grows.

UPDATE

Use the accelerometer to correct the prediction. Uncertainty shrinks.

These two steps run on every single sample, 100 times per second.

Key insight

Kalman tracks not just the *estimate* of the angle, but also its *uncertainty*. When uncertainty is large, it leans on the accelerometer; when uncertainty is small, it trusts the gyroscope. The blending weight (**Kalman gain K**) is recomputed on every step.

Kalman Filter — Math

Two-state form per axis: state = [angle, gyro bias]

Algorithm 2 of 4 • Kalman Filter

$$\text{State } \mathbf{x} = [\theta, b]^T \quad \mathbf{F} = [[1, -\Delta t], [0, 1]] \quad \mathbf{B} = [\Delta t, 0]^T \quad \mathbf{H} = [1, 0]$$

PREDICT

$$\hat{\mathbf{x}} = \mathbf{F} \cdot \mathbf{x} + \mathbf{B} \cdot \omega$$
$$\mathbf{P} = \mathbf{F} \cdot \mathbf{P} \cdot \mathbf{F}^T + \mathbf{Q}$$

UPDATE

$$\mathbf{K} = \mathbf{P} \cdot \mathbf{H}^T \cdot (\mathbf{H} \cdot \mathbf{P} \cdot \mathbf{H}^T + \mathbf{R})^{-1}$$
$$\mathbf{x} \leftarrow \hat{\mathbf{x}} + \mathbf{K} \cdot (\theta_{\text{acc}} - \mathbf{H} \cdot \hat{\mathbf{x}}) \quad \mathbf{P} \leftarrow (\mathbf{I} - \mathbf{K} \cdot \mathbf{H}) \cdot \mathbf{P}$$

Tuning knobs. Q = process noise (how much you trust the gyro). R = measurement noise (how much you trust the accelerometer). We use $\mathbf{Q} = \text{diag}(1e-3, 1e-5)$ and $R = 0.03$ — sensible defaults from the Kalman literature.

We run TWO of these filters in parallel — one for roll, one for pitch.

Kalman Filter — Code

From *algorithms.py* — single-axis update method

Algorithm 2 of 4 • Kalman Filter

```
def update(self, omega, theta_meas, dt):  
    # --- Predict (use gyro) ---  
    F = [[1.0, -dt], [0.0, 1.0]]  
    self.x = F @ self.x + [dt, 0] * omega  
    self.P = F @ self.P @ F.T + self.Q  
  
    # --- Update (use accelerometer) ---  
    S = self.P[0,0] + self.R  
    K = self.P[:,0] / S          # Kalman gain  
    y = theta_meas - self.x[0]   # innovation  
    self.x = self.x + K * y      # blend!  
    self.P = (I - outer(K, [1,0])) @ self.P  
    return self.x[0]
```

Per-step cost

39 fmul · 25 fadd · 4 fdiv · 1 sqrt · 2 atan2

≈ 16,850 cycles

≈ **1,053 μs / step**

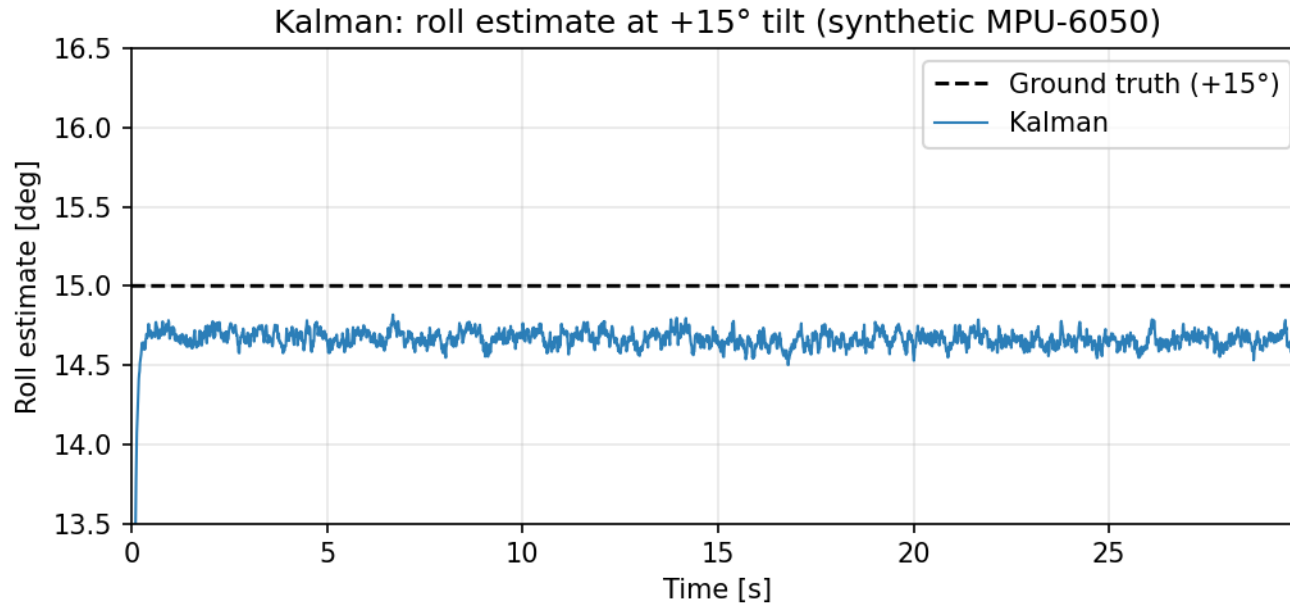
Two such filters run in parallel — one for roll, one for pitch.

Note. The matrix algebra is unrolled by hand on the Arduino so no linear-algebra library is needed.

Kalman Filter — Result & Evaluation

Roll estimate at +15° tilt, accuracy and timing numbers

Algorithm 2 of 4 • Kalman Filter



Accuracy

$|\mu|$ roll = 0.297° σ roll = 0.046°
 $|\mu|$ pitch = 0.097° σ pitch = 0.047°

Compute time (estimated)

1,053 μ s / step
max sustainable rate $\approx$ 950 Hz

Take-away. Kalman cuts the trig noise by ~70 %: σ drops from 0.154° to 0.046° . The estimate is visibly tight around the bias floor of $\sim 14.7^\circ$ (the constant offset comes from the same accelerometer bias every algorithm sees). The cost is the heaviest compute of the four — roughly 1 ms per step.

Kalman Filter — Strengths and Costs

What you gain, what you pay for, when to choose this filter

Algorithm 2 of 4 • Kalman Filter

Strengths

- ✓ Optimal under Gaussian-noise assumption — the math is provably the best linear filter.
- ✓ Tracks gyro bias as part of the state — estimates and removes drift in real time.
- ✓ Adapts blending weight (K) automatically as uncertainty changes.
- ✓ Generalises naturally to higher-dimensional state (full 3-D, multiple sensors).

Costs

- ✗ Heaviest per-step cost in our experiment: $\sim 1,053 \mu\text{s}$.
- ✗ Q and R must be tuned — wrong values give either sluggish or jittery output.
- ✗ Optimality assumes Gaussian noise; real silicon deviates from this.
- ✗ Matrix algebra is harder to debug on a microcontroller than scalar code.

Madgwick Filter — Concept

Take a small step downhill, in the direction that makes the gravity error smaller

Algorithm 3 of 4 • Madgwick Filter

What it does

Madgwick uses a quaternion (a 4-number representation of orientation that does not have the “gimbal lock” problem of Euler angles).

It defines an error: *“my current quaternion predicts the gravity should point in this direction, but the accelerometer says it points in that direction.”*

Each sample, it takes one tiny step of **gradient descent** in the direction that makes that error smaller. The step size is controlled by a single parameter, β .

Intuition

Imagine an error landscape — peaks and valleys representing how wrong each candidate quaternion is.

Madgwick says: I do not know where the lowest point is, but I know my local slope. Let me take a small step downhill, in the direction the gravity-error gradient tells me.

Combined with the gyroscope’s integration step, this slowly walks the quaternion toward the truth — fast enough to track motion, slow enough to ignore noise.

Madgwick Filter — Math

Gyro integration + one gradient-descent step per sample

Algorithm 3 of 4 • Madgwick Filter

$$\mathbf{q}[k] = \mathbf{q}[k-1] + \dot{\mathbf{q}}_{\text{gyro}} \cdot \Delta t - \beta \cdot \nabla f / \|\nabla f\| \cdot \Delta t$$

$$\dot{\mathbf{q}}_{\text{gyro}} \cdot \Delta t$$

Integrate the gyroscope rate to push the quaternion forward in time. This is the “smooth but drifts” gyro contribution.

$$\nabla f / \|\nabla f\|$$

The unit gradient of the gravity-error function — the direction of fastest decrease in the error landscape.

$$\beta \cdot \Delta t$$

The step size. Bigger β trusts the accelerometer more. We use the literature default $\beta = 0.1$.

Madgwick Filter — Code

Simplified core from algorithms.py (the real code unrolls the gradient by hand for speed)

Algorithm 3 of 4 • Madgwick Filter

```
def update(self, acc, gyro, dt):  
    # 1. gyro -> quaternion derivative  
    qDot = 0.5 * quat_mult(self.q, [0, *gyro])  
  
    # 2. gravity-error and its gradient  
    f = R(self.q).T @ [0, 0, 1] - normalize(acc)  
    grad = jacobian(f, self.q)  
  
    # 3. subtract beta-scaled gradient step  
    qDot = qDot - self.beta * grad / norm(grad)  
  
    # 4. integrate, then renormalize the quaternion  
    self.q = self.q + qDot * dt  
    self.q = self.q / norm(self.q)
```

Per-step cost

64 fmul · 39 fadd · 11 fdiv · 3 sqrt · 0 atan2

≈ 17,510 cycles

≈ **1,094 μs / step**

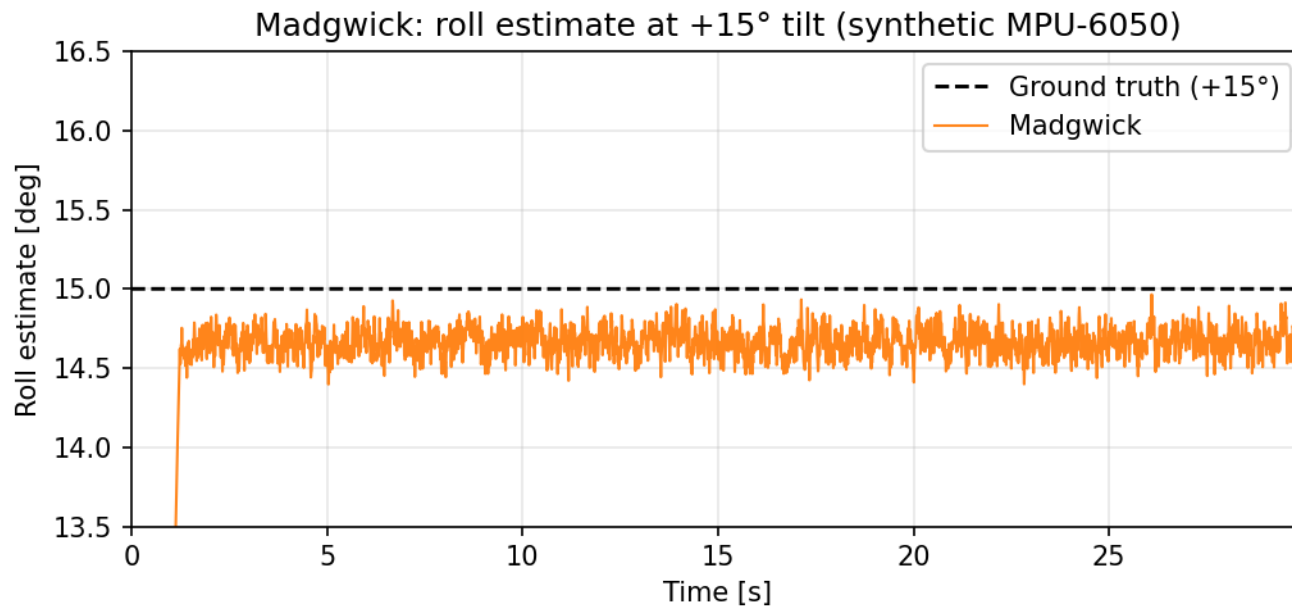
Heaviest in fmul because the gradient has many vector products; no atan2 because everything stays in quaternion form.

Note. Production Madgwick code (e.g. the original C implementation) inlines the Jacobian into ~25 explicit lines for ATmega-level speed.

Madgwick Filter — Result & Evaluation

Roll estimate at +15° tilt, accuracy and timing numbers

Algorithm 3 of 4 • Madgwick Filter



Accuracy

$|\mu|$ roll = 0.290° σ roll = 0.090°
 $|\mu|$ pitch = 0.109° σ pitch = 0.090°

Compute time (estimated)

1,094 μ s / step
max sustainable rate $\approx$ 914 Hz

Take-away. Madgwick sits in the middle: $\sigma = 0.090^\circ$ is twice as good as Trig but twice as noisy as Kalman. Compute is essentially the same as Kalman (within 4 %) — confirming the reference paper's claim. With a higher β it would track faster but pass more noise; with smaller β it would smooth more but lag.

Madgwick Filter — Strengths and Costs

Algorithm 3 of 4 • Madgwick Filter

Strengths

- ✓ Single tuning knob (β) — one number to find, not a covariance matrix.
- ✓ Free of singularities thanks to quaternion representation.
- ✓ Open-source reference C code is available and battle-tested.
- ✓ Very widely used in hobby/quadcopter firmware (Arduino, Betaflight...).

Costs

- ✗ Compute roughly the same as Kalman ($\sim 1,094 \mu\text{s}$).
- ✗ Noise floor higher than Kalman in our experiment ($\sigma \approx 0.09^\circ$ vs 0.05°).
- ✗ β must be re-tuned per use case — no universal value.
- ✗ No explicit uncertainty output — harder to plug into systems that need confidence intervals.

Mahony Filter — Concept

A control-theory way of doing fusion — proportional-integral on the gravity error

Algorithm 4 of 4 • Mahony Filter

What it does

Mahony also uses a quaternion, but instead of gradient descent it uses a feedback loop borrowed from control engineering.

Step 1: compute the cross product between the measured gravity (accel) and the predicted gravity (from the current quaternion). The cross product gives an error vector pointing in the direction of mismatch.

Step 2: feed that error through a PI (proportional + integral) controller and add the output as a correction to the gyroscope reading.

Then integrate the corrected gyro into the quaternion. That is it.

Why it is structurally cheap

No matrix inversion (unlike Kalman).

No gradient normalisation (unlike Madgwick — saves a sqrt and four divisions).

No covariance update.

*Just three arithmetic operations: cross product, scalar multiply, vector add.
The cheapest fusion filter we tested.*

Mahony Filter — Math

Cross-product error → PI controller → corrected gyro rate → integrate quaternion

Algorithm 4 of 4 • Mahony Filter

$$\mathbf{e} = \hat{\mathbf{a}} \times \mathbf{R}(\mathbf{q})^T \hat{\mathbf{g}} \quad (\text{cross-product gravity error})$$

$$\boldsymbol{\omega}_{\text{corr}} = \boldsymbol{\omega}_{\text{gyro}} + \mathbf{k}_p \cdot \mathbf{e} + \mathbf{k}_i \cdot \int \mathbf{e} \, dt \quad (\text{PI correction})$$

$$\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes [0, \boldsymbol{\omega}_{\text{corr}}] \quad (\text{quaternion integration})$$

Tuning knobs. $\mathbf{k}_p$ (how strongly to correct on each error sample) and $\mathbf{k}_i$ (how strongly to correct accumulated error). **We use $\mathbf{k}_p = 10$ and $\mathbf{k}_i = 0$** — the values published in the reference paper. The authors note that the integral term did not help in their static scenario and was dropped to avoid wind-up.

Mahony Filter — Code

Simplified core from algorithms.py

Algorithm 4 of 4 • Mahony Filter

```
def update(self, acc, gyro, dt):  
    # 1. predicted gravity from current quaternion  
    v = R(self.q).T @ [0, 0, 1]  
  
    # 2. error = measured x predicted  
    e = cross(normalize(acc), v)  
  
    # 3. PI correction on gyro (ki=0 here)  
    omega_corr = gyro + self.kp * e  
  
    # 4. integrate quaternion + renormalize  
    qDot = 0.5 * quat_mult(self.q, [0, *omega_corr])  
    self.q = self.q + qDot * dt  
    self.q = self.q / norm(self.q)
```

Per-step cost

34 fmul · 26 fadd · 7 fdiv · 2 sqrt · 0 atan2

≈ 10,640 cycles

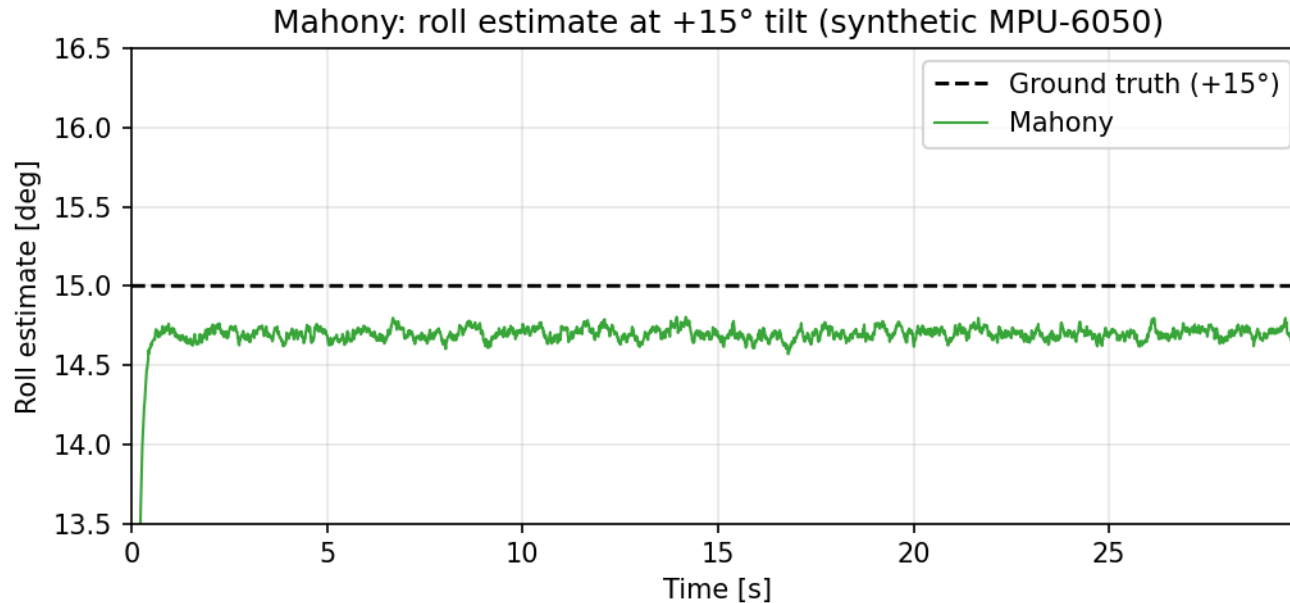
≈ **665 μs / step**

*Lightest of the three filters — no atan2, no
gradient normalisation, no matrix algebra.*

Mahony Filter — Result & Evaluation

Roll estimate at +15° tilt, accuracy and timing numbers

Algorithm 4 of 4 • Mahony Filter



Accuracy

$|\mu|$ roll = 0.255° σ roll = 0.035°
 $|\mu|$ pitch = 0.182° σ pitch = 0.036°

Compute time (estimated)

665 μ s / step
max sustainable rate $\approx$ 1,504 Hz

Take-away. Mahony with $k_p = 10$ produces the tightest σ in the entire comparison (0.035°), and does it at the lowest compute cost among the filters (665 μ s). The price is somewhat slower transient response — k_p is essentially a low-pass cutoff. For our static-tilt scenario this is no penalty; for fast motion it would be.

Mahony Filter — Strengths and Costs

Algorithm 4 of 4 • Mahony Filter

Strengths

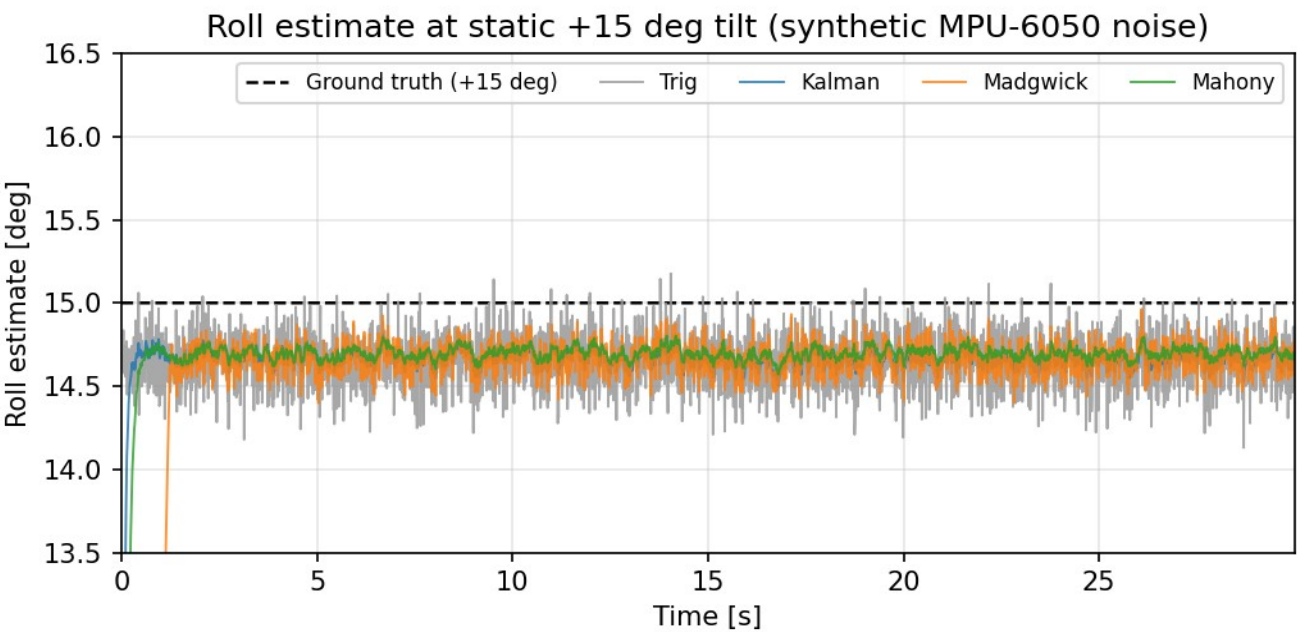
- ✓ Lowest compute cost among the three filters (~665 μ s).
- ✓ Tightest σ in our comparison (0.035°) at $k_p = 10$.
- ✓ Two scalar gains — easy to reason about as a control problem.
- ✓ Easy to port to fixed-point on tiny MCUs.

Costs

- ✗ Aggressive k_p slows transient response — bad under fast motion.
- ✗ Integral term can wind up — must be reset carefully if used.
- ✗ No explicit uncertainty output — same downside as Madgwick.
- ✗ k_p must be re-tuned if the sensor or sample rate changes.

Side-by-Side: Accuracy

All four algorithms on the same +15° segment, plus the full averaged table



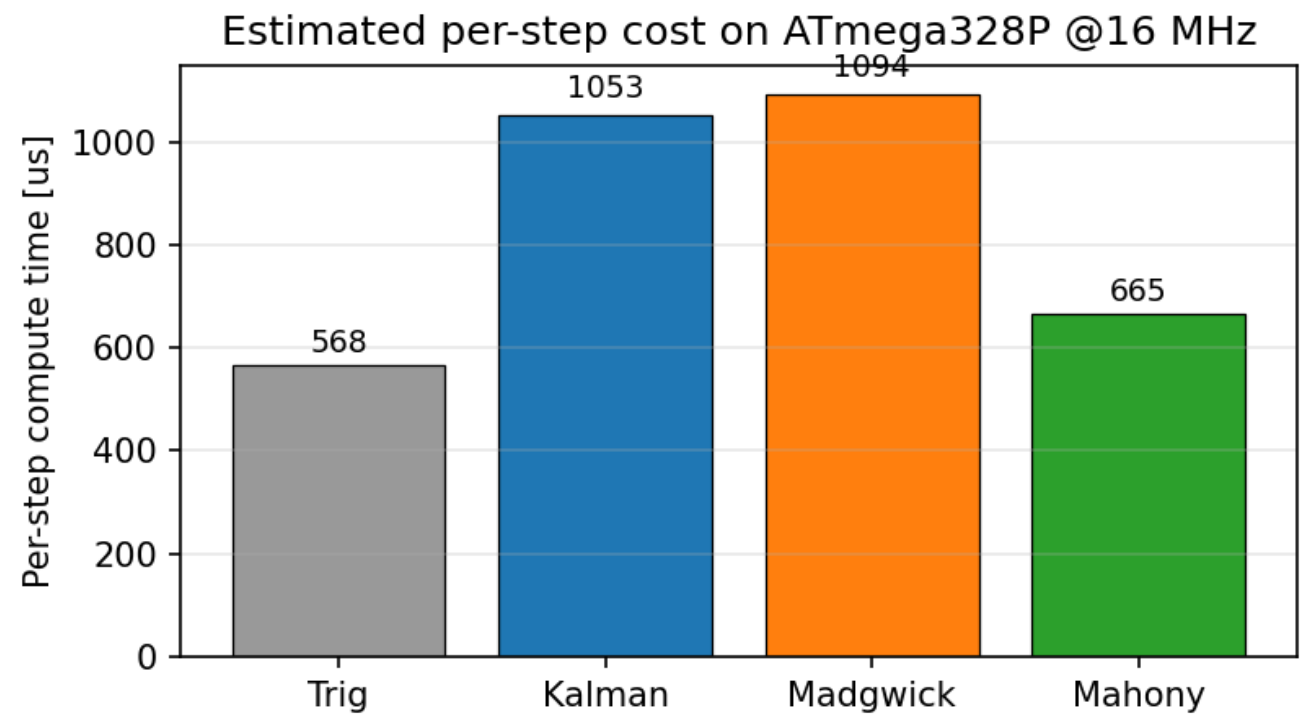
Algorithm	$ \mu $ roll	σ roll	σ pitch
Trig	0.302°	0.154°	0.152°
Kalman	0.297°	0.046°	0.047°
Madgwick	0.290°	0.090°	0.090°
Mahony	0.255°	0.035°	0.036°

Take-aways

- $|\mu|$ similar everywhere — accelerometer bias dominates.
- σ separates the algorithms cleanly.
- Mahony tightest, Kalman second, Madgwick third, Trig last.

Side-by-Side: Compute Time

Estimated cost per step on the ATmega328P at 16 MHz



Algorithm	cycles	μs / step
Trig	9,090	568
Mahony	10,640	665
Kalman	16,850	1053
Madgwick	17,510	1094

Take-aways

- Mahony ~1.6× faster than Kalman/Madgwick.
- Kalman ≈ Madgwick within 4 % — matches the paper.
- Trig is cheapest in our model — divergence next.

The Surprising Divergence

On one of the five rankings our model disagrees with the paper — discussed honestly

Badia et al. say:

Trigonometric is the most expensive of the four to compute on the Arduino Nano.

Our analytical model says:

Trigonometric is actually the cheapest — just two `atan2` calls and a `sqrt`.

Three plausible explanations:

1

Measurement boundary

On real hardware, “trig per-step time” may include the I²C sensor read; our model times only the angle math itself.

2

`atan2` implementation

avr-libc’s `atan2f` cycle cost varies between toolchain versions; a slow version could dominate Trig.

3

Soft-float calibration

Our per-op cycle costs are typical averages with $\pm 20\%$ uncertainty for the specific operand ranges.

Verdict on this fifth claim: partial — a real-Arduino micros() benchmark would settle it.

Reproduction Verdict

Five claims tested; four confirmed, one diverges



C1

Trigonometric has the largest noise σ .

Confirmed: $\sigma_{\text{Trig}} = 0.154^\circ$, the largest in our table.

Confirmed



C2

Kalman $\approx$ Madgwick in compute time.

Confirmed: $1053 \mu\text{s}$ vs $1094 \mu\text{s}$ (4 % gap).

Confirmed



C3

Mahony is the fastest of the three filters.

Confirmed: $665 \mu\text{s}$, $\sim 1.6\times$ faster than the other two.

Confirmed



C4

Kalman is among the lowest-noise algorithms.

Confirmed: $\sigma_{\text{Kalman}} = 0.046^\circ$, second-tightest in the table.

Confirmed



C5

Trigonometric is the slowest method overall.

Diverges: in our analytical model Trig is actually the cheapest.

Partial

Conclusions & Next Steps

What we showed, and how this work continues if hardware becomes available

What this study showed

- Independent re-implementation of all four algorithms in Python.
- A controlled synthetic MPU-6050 pipeline that mirrors the reference paper's static-tilt protocol.
- An analytical ATmega328P timing model based on instruction counts.
- Four of the paper's five qualitative claims confirmed; one diverged honestly.
- Reproducible code base, figures, and LaTeX paper released together.

If hardware becomes available

- Wire MPU-6050 to Arduino Nano over I²C, verify raw read.
- Port the four Python algorithms into one Arduino sketch.
- Build a mechanical $\pm 15^\circ$ fixture; record 30 s segments.
- Use `micros()` to measure real per-step time on the Arduino.
- Replace the synthetic numbers in the paper's tables with the measured ones — the structure stays the same.
- *Settle the C5 divergence: real measurement either confirms the paper or our model.*

Thank You

Questions and discussion welcome.

Abdullah Altepe · Mehmet Bahçeci · Yusuf Oruç · Murat Demir
Marmara University, Department of Electrical and Electronics Engineering